

GLAM Security Enhancements Audit Report

Table of Contents

1. Executive Summary

About GLAM

Audit Summary

Overall Security Posture

After Fix Review Summary

Before Fix Review Summary

Launch Recommendations

Audit Scope

4

4

4

4

4

5

5

5

2. Assumptions and Considerations

Audit Assumptions

6

6

3. Severity Definitions

Impact

Likelihood

Severity Classification Matrix

7

7

7

7

4. Findings

L01: Duplicate-mint oracle fix remains incomplete for Kamino reserves and aggregate Scope oracles

L02: Cross-denomination normalization preserves confidence in the wrong units

L03: Kamino liquidation can realize reserve mints outside the vault pricing set

9

9

13

17

5. Enhancement Opportunities

E01: Split oracle and execution slippage limits in swap_handler_v2

20

20

About Us

About Adevar Labs

Audit Methodology

Confidentiality Notice

21

21

21

23

Legal Disclaimer	23
Appendix A: Hashes of program binaries	24

1. Executive Summary

About GLAM

GLAM is a protocol for programmable investment infrastructure on Solana. It provides composable primitives like vaults, mints, integrations, policies, and permissions to launch tokenized products and onchain strategies with precision and control.

Audit Summary

This review focused on recent changes around swap execution, trusted external price feeds, Kamino integrations, and shared permission refactors. We also kept a close eye on how these changes could affect AUM and NAV reporting.

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	3	3	0

Enhancement opportunities: 1.

Overall Security Posture

The reviewed branch reflects a clear effort to strengthen the protocol’s security model. The main changes are intended to improve price safety during execution, tighten default permissions, make integration behavior more consistent, and reduce the chance of unexpected asset or policy bypasses. These are directionally strong improvements for the platform.

After Fix Review Summary

The previously identified issues were addressed and the updated code now aligns much more closely with the intended security model. The reviewed fixes materially improved pricing correctness, policy enforcement, and validation consistency across the affected Kamino, Jupiter, and oracle-related flows.

Overall, the post-fix code presents a meaningfully stronger and more consistent security posture than the pre-fix state, with the main security concerns from the original review now resolved.

Before Fix Review Summary

The work covered meaningful security-sensitive changes: Jupiter Swap v2 with quote checks against external prices, a deny-by-default swap policy model, new oracle types, new Kamino instruction handlers, and a broad permission refactor across integration programs. The main strengths were better use of price data, clearer policy structure, and more consistent validation patterns across programs.

Before fixes, the main remaining issues were concentrated in a few themes rather than being spread evenly across the codebase. The primary concerns were around oracle-based price validation and a small number of permission and asset-accounting paths that did not yet fully match the stricter control model introduced by the refactor.

Launch Recommendations

Address the remaining medium-severity issues before launch or broad rollout, especially the ones affecting pricing accuracy, order validation, and policy enforcement. We also recommend validating that the documented assumptions and considerations remain accurate in practice, and implementing the identified enhancement.

Audit Scope

- **Repository:** <https://github.com/glamsystems/glam>
- **Before Fix Review Commit Hash:** [79bdd1f9387c96512f18e40079c6577a60dc396d](#)
- **After Fix Review Commit Hash:** [ce797d3e4beb44f2944461dea7aa3f1c02c9603a](#)
- **Exclusions:**
 - [anchor/programs/ext_drift](#)

2. Assumptions and Considerations

Audit Assumptions

- **Borrow permission currently appears to include Kamino elevation-group selection.** The current Kamino extension appears to treat **request_elevation_group** as part of existing borrow authority, subject to market allowlisting. Elevation groups can materially affect borrowing conditions, including LTV, liquidation thresholds, loan enablement, and debt reserve requirements. This is presented as a design consideration rather than a finding. If this model is intentional, stakeholders should be comfortable that granting **Borrow** also allows moving the vault into any more permissive elevation group supported by the selected Kamino market.
- **token_close_account** and **reclaim_rent** currently behave as intentional exceptions to the deny-by-default integration model, remaining owner-or-delegate controlled rather than protocol-permission gated.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

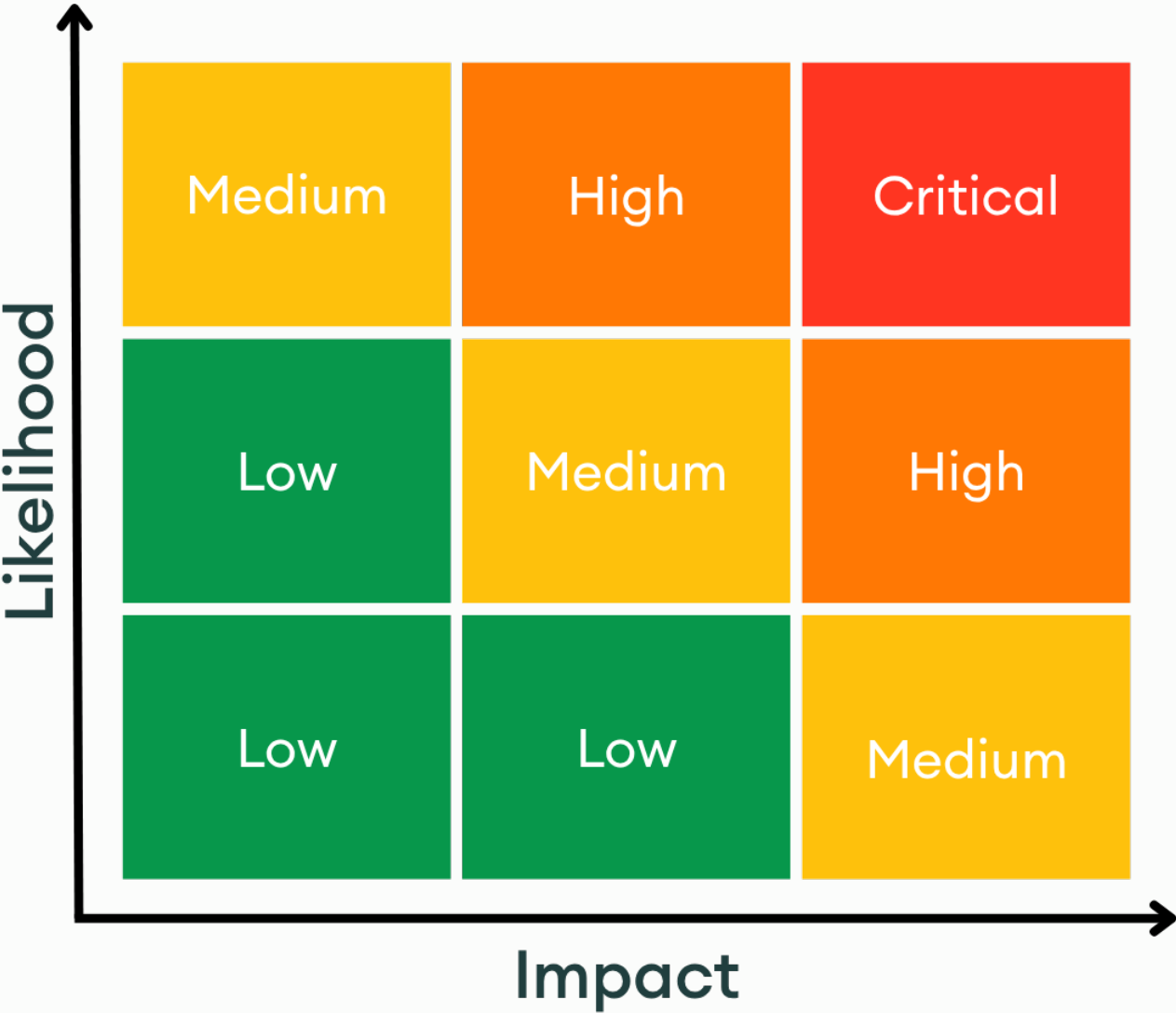
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability

- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

L01: Duplicate-mint oracle fix remains incomplete for Kamino reserves and aggregate Scope oracles



LOCATION

glam/79bdd1f9/anchor/programs/glam_protocol/src/state/price/oracle.rs:L855 [↗](#)

RELEVANT SNIPPET

>_oracle.rs

RUST

```
853: }
854: let idx = asset metas_ref
855: .find_by_asset(mint)
856: .ok_or(GlamError::InvalidPricingOracle)?;
857: let max_age_seconds = oracle_configs
```

DESCRIPTION

The code correctly handles the direct-oracle path by matching asset metadata using (`mint, oracle_account.key()`). However, the same duplicate-mint problem remains in two other branches of `validate_oracle`.

In the Kamino reserve branch, the code verifies that the reserve's `liquidity_mint` matches the requested mint, but it still resolves the asset-meta row with `find_by_asset(mint)`. If the same mint is configured against multiple Kamino reserves, validation will still bind to the first row for that mint rather than the row whose `asset_meta.oracle` matches the supplied reserve account.

In the aggregate Scope fallback, the code likewise falls back to `find_by_asset(mint)` before validating the mapped oracle/feed. This means duplicate-mint Scope entries can still be matched against the first row only, even when `agg_index` points to a different configured feed. Because the selected row is also used to derive metadata, the wrong `max_age_seconds` can be returned, and any vault-local override for the intended row can be missed.

So the code handles the first duplicate-mint occurrence, but not the Kamino or aggregate Scope occurrences.

PRACTICAL IMPACT

Although the code does duplicate-mint handling for direct oracle accounts, Kamino reserve validation and aggregate Scope validation can still resolve the first asset meta for a mint instead of the intended **(mint, oracle)** row. In the aggregate path, this also means the correct local **max_age_seconds** override may be skipped.

RECOMMENDATION

Complete the duplicate-mint fix by resolving the matched asset-meta row with **(mint, oracle)** in the remaining paths as well. For Kamino reserves, match against **oracle_account.key()**. For aggregate Scope oracles, first resolve the mapped oracle/feed from the mapping account, then match on **(mint, mapped_oracle)** and derive **max_age_seconds** from that matched row. Example fix:

DIFF

```
diff --git a/anchor/programs/glam_protocol/src/state/price/oracle.rs
      b/anchor/programs/glam_protocol/src/state/price/oracle.rs
index 45d91a9..43d530b 100644
--- a/anchor/programs/glam_protocol/src/state/price/oracle.rs
+++ b/anchor/programs/glam_protocol/src/state/price/oracle.rs
@@ -306,19 +306,15 @@ impl Scope {
     })
 }

-    /// Validates oracle and mapping accounts, returns the feed id or oracle pubkey at the
-    /// given index
-    pub fn validate_oracle_mapping(
+    /// Reads the feed id or oracle pubkey at the given index after validating the
+    /// scope prices and mapping accounts line up.
+    pub fn get_mapped_oracle(
+        scope_prices: &AccountInfo, // scope prices account, points to oracle mapping account
+        and contains prices array
+        agg_oracle_mapping: &AccountInfo, // oracle mapping account, contains feed ids or
+        oracle pubkeys
+        expected_feed_id_or_oracle: Pubkey,
+        agg_index: i16, // index in glam global config
+    ) -> Result<()> {
+    +    agg_index: i16, // index in glam global config
+    +    ) -> Result<Pubkey> {
+        require!(agg_index >= 0, GlamError::InvalidPricingOracle);

-        // In this function we don't parse the prices from scope_prices account yet
-        // Oracle mappings account has an array of priceInfoAccounts, using agg_index we can
-        // find a pubkey, which is a feed id or an oracle address
-        // We validate the feed id or oracle address matches the expected feed id or oracle
-        // address tracked in asset metas
-
+        require_keys_eq!(
+            *scope_prices.owner,
+            Self::PROGRAM_ID,
@@ -352,7 +348,21 @@ impl Scope {

+        let offset = Self::ORACLE_MAPPING_DISCRIMINATOR.len() + agg_index as usize * 32;
+        require!(offset + 32 <= data.len(), GlamError::InvalidPricingOracle);
-        let feed_id_or_oracle = read_pubkey!(data, offset);
```

... continued

DIFF

```

+   Ok(read_pubkey!(data, offset))
+ }
+
+ /// Validates oracle and mapping accounts, returns the feed id or oracle pubkey at the
+ given index
+ pub fn validate_oracle_mapping(
+     scope_prices: &AccountInfo, // scope prices account, points to oracle mapping account
+     and contains prices array
+     agg_oracle_mapping: &AccountInfo, // oracle mapping account, contains feed ids or
+     oracle pubkeys
+     expected_feed_id_or_oracle: Pubkey,
+     agg_index: i16, // index in glam global config
+ ) -> Result<()> {
+     // In this function we don't parse the prices from scope_prices account yet.
+     // Oracle mappings account has an array of priceInfoAccounts; using agg_index we can
+     find
+     // the feed id or oracle address and validate it matches the asset meta config.
+     let feed_id_or_oracle =
+         Self::get_mapped_oracle(scope_prices, agg_oracle_mapping, agg_index)?;
+     require_keys_eq!(
+         feed_id_or_oracle,
+         expected_feed_id_or_oracle,
@@ -852,7 +862,7 @@ pub fn validate_oracle(
+         require_keys_eq!(liquidity_mint, *mint, GlamError::InvalidPricingOracle);
+     }
+     let idx = asset_metas_ref
-         .find_by_asset(mint)
+         .find_by_asset_and_oracle(mint, &oracle_account.key())
+         .ok_or(GlamError::InvalidPricingOracle)?;
+     let max_age_seconds = oracle_configs
+         .and_then(|oc| oc.get_max_age_seconds_by_index(idx as u16))
@@ -883,25 +893,23 @@ pub fn validate_oracle(
+     // - oracle_account is owned by scope program and is the scope prices account
+     // - oracle_account.oracle_mapping == agg_oracle_mapping.key
+     // - asset_meta.oracle == agg_oracle_mapping[idx]
-     let idx = asset_metas_ref
-     .find_by_asset(mint)
-     .ok_or(GlamError::InvalidPricingOracle)?;
+     require!(
+         agg_index >= 0 && agg_oracle_mapping.is_some(),
+         GlamError::InvalidPricingOracle
+     );
+     let agg_oracle_mapping = agg_oracle_mapping.unwrap();
-     Scope::validate_oracle_mapping(
-         oracle_account,
-         agg_oracle_mapping,
-         asset_metas_ref.get_oracle(idx)?,
-         agg_index,
-     )?;
+     let mapped_oracle = Scope::get_mapped_oracle(oracle_account, agg_oracle_mapping,
+     agg_index)?;
+     let idx = asset_metas_ref
+     .find_by_asset_and_oracle(mint, &mapped_oracle)

```

... continued

DIFF

```
+      .ok_or(GlamError::InvalidPricingOracle)?;
+      let max_age_seconds = oracle_configs
+      .and_then(|oc| oc.get_max_age_seconds_by_index(idx as u16))
+      .unwrap_or(asset metas_ref.get_max_age_seconds(idx)?);

      Ok((
        asset_metas_ref.get_oracle_source(idx)?,
        asset_metas_ref.get_decimals(idx)?,
-       asset_metas_ref.get_max_age_seconds(idx)?,
+       max_age_seconds,
      ))
    }
  }
```

FIX REVIEW NOTES

Kamino and aggregate Scope now match asset meta by (mint, oracle), fixing duplicate-mint selection.

L02: Cross-denomination normalization preserves confidence in the wrong units



LOCATION
glam/79bdd1f9/anchor/programs/glam_protocol/src/instructions/jupiter_swap_v2.rs:L64 [↗](#)

RELEVANT SNIPPET

```
>_jupiter_swap_v2.rsRUST
62: OraclePriceData {
63:     price: input_usd_price,
64:     confidence: input_price.confidence,
65:     denom: PriceDenom::USD,
66:     delay: input_price.delay,
```

DESCRIPTION

When `normalize_prices` converts a SOL-denominated price into USD (and vice versa), it correctly rescales the `price` field but copies `confidence` unchanged. This breaks the implicit invariant used elsewhere in the codebase that confidence is an absolute interval in the same denomination as price, as seen in `validate_age_and_confidence`.

Example:

Stage	Price	Confidence	Feed
Original SOL-denom	1.10 SOL	0.01 SOL	1.10 SOL +/- 0.01 SOL
Correct after USD conversion	\$220	\$2	\$220 +/- \$2
Current code returns	\$220	\$10	\$220 +/- \$10

RECOMMENDATION

When converting a SOL-denominated price to USD, convert confidence into the same denomination as price instead of copying it verbatim. Example fix:

DIFF

```
diff --git a/anchor/programs/glam_protocol/src/instructions/jupiter_swap_v2.rs
    b/anchor/programs/glam_protocol/src/instructions/jupiter_swap_v2.rs
index e4d6f4e..3cb9635 100644
--- a/anchor/programs/glam_protocol/src/instructions/jupiter_swap_v2.rs
+++ b/anchor/programs/glam_protocol/src/instructions/jupiter_swap_v2.rs
@@ -28,6 +28,61 @@ use crate::{
    utils::{safe_cast::Cast, safe_math::SafeMath},
};

+/// Converts a SOL-denominated oracle price into USD units using the SOL/USD oracle.
+///
+/// The composed confidence interval conservatively carries both the SOL-denominated
+/// leg and the SOL/USD leg:
+///  $(a \pm ca) * (b \pm cb) = ab \pm (a*cb + b*ca + ca*cb)$ .
+fn convert_sol_price_to_usd(
+    sol_denom_price: &OraclePriceData,
+    sol_usd_price: &OraclePriceData,
+) -> Result<OraclePriceData> {
+    require_eq!(
+        sol_denom_price.denom,
+        PriceDenom::SOL,
+        GlamError::InvalidPricingOracle
+    );
+    require_eq!(
+        sol_usd_price.denom,
+        PriceDenom::USD,
+        GlamError::InvalidPricingOracle
+    );
+
+    let price = sol_denom_price
+        .price
+        .cast::<u128>()?
+        .safe_mul(sol_usd_price.price.cast()?)?
+        .safe_div(SOL_PRICE_PRECISION.cast()?)?
+        .cast::<u64>()?;
+
+    let sol_leg_confidence = sol_denom_price
+        .confidence
+        .cast::<u128>()?
+        .safe_mul(sol_usd_price.price.cast()?)?
+        .safe_div(SOL_PRICE_PRECISION.cast()?)?;
+    let sol_usd_leg_confidence = sol_denom_price
+        .price
+        .cast::<u128>()?
+        .safe_mul(sol_usd_price.confidence.cast()?)?
+        .safe_div(SOL_PRICE_PRECISION.cast()?)?;
+    let interaction_confidence = sol_denom_price
+        .confidence
+        .cast::<u128>()?

```

... continued

DIFF

```

+     .safe_mul(sol_usd_price.confidence.cast())?
+     .safe_div(SOL_PRICE_PRECISION.cast())?;
+   let confidence = sol_leg_confidence
+     .safe_add(sol_usd_leg_confidence)?
+     .safe_add(interaction_confidence)?
+     .cast::<u64>()?;
+
+   Ok(OraclePriceData {
+     price,
+     confidence,
+     denom: PriceDenom::USD,
+     delay: sol_denom_price.delay.max(sol_usd_price.delay),
+   })
+ }
+
+ /// Normalizes two oracle prices to a common denomination for comparison.
+ ///
+ /// - LST-to-LST: Keep SOL denomination (both prices already in SOL)
+ @@ -49,42 +104,17 @@ fn normalize_prices(
+   // Input is SOL-denominated (LST), output is USD - convert input to USD
+   (PriceDenom::SOL, PriceDenom::USD) => {
+     let sol_price = sol_usd_price.ok_or(GlamError::MissingAccount)?;
+     - // input_price is in SOL (e.g., 1.1 SOL per mSOL with precision 10^9)
+     - // sol_price is in USD (e.g., $200 per SOL with precision 10^6)
+     - // Result should be in USD with precision 10^6
+     let input_usd_price = input_price
+     - .price
+     - .cast::<u128>()?
+     - .safe_mul(sol_price.price.cast())?
+     - .safe_div(SOL_PRICE_PRECISION.cast())?
+     - .cast::<u64>()?;
+     Ok((
+     -   OraclePriceData {
+     -     price: input_usd_price,
+     -     confidence: input_price.confidence,
+     -     denom: PriceDenom::USD,
+     -     delay: input_price.delay,
+     -   },
+     +   convert_sol_price_to_usd(input_price, sol_price)?,
+     *output_price,
+   ))
+ }
+ // Input is USD, output is SOL-denominated (LST) - convert output to USD
+ (PriceDenom::USD, PriceDenom::SOL) => {
+   let sol_price = sol_usd_price.ok_or(GlamError::MissingAccount)?;
+   - let output_usd_price = output_price
+   - .price
+   - .cast::<u128>()?
+   - .safe_mul(sol_price.price.cast())?
+   - .safe_div(SOL_PRICE_PRECISION.cast())?
+   - .cast::<u64>()?;
+   Ok((
+     *input_price,

```

... continued

DIFF

```
-         OraclePriceData {  
-             price: output_usd_price,  
-             confidence: output_price.confidence,  
-             denom: PriceDenom::USD,  
-             delay: output_price.delay,  
-         },  
+         convert_sol_price_to_usd(output_price, sol_price)?,  
    ))  
  }  
}
```

FIX REVIEW NOTES

Fixed according to the recommendation. In current usage, this path is only reached for SOL-denominated LST functional pricing, where confidence is always zero, so the unit mismatch does not affect live behavior. The concern is forward-looking and would only become relevant if a future release routes non-USD or nonzero-confidence oracle prices through the same path without additional guards.

L03: Kamino liquidation can realize reserve mints outside the vault pricing set



LOCATION

glam/79bdd1f9/anchor/programs/ext_kamino/src/instructions/lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs:L104-L109 [↗](#)

RELEVANT SNIPPET

```
>_ lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs RUST
102:         .build_remaining_accounts(ctx.remaining_accounts);
103:
104:     ctx.accounts.execute_cpi(
105:         data,
106:         remaining_accounts,
107:         vec![],
108:         integration_authority_signer_seeds,
109:     )
110: }
111:
```

DESCRIPTION

The Kamino borrow wrapper applies mint-level gating before funds can reach a vault ATA: it checks the optional state-level borrowable set and the Kamino **borrow_allowlist** against **borrow_reserve_liquidity_mint** in [lending_borrow_obligation_liquidity_v2.rs](#). The liquidation wrapper does not apply any analogous mint-level check. In [lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs](#) it only requires that the lending market is allowlisted and that the caller has liquidation permission.

```
>_ lending_borrow_obligation_liquidity_v2.rs RUST
22:
23: impl<'info> LendingBorrowObligationLiquidityV2<'info> {
24:     fn check_access(&self) -> Result<()> {
25:         // Check global borrowable allowlist if exists
26:         let state = &self.glam_state;
```

```
>_lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs
```

RUST

```
21:
22: impl<'info> LendingLiquidateObligationAndRedeemReserveCollateralV2<'info> {
23:     fn check_access(&self) -> Result<()> {
24:         require!(
25:             self.load_policy()?
```

The liquidation instruction still forwards the selected reserve mints and vault-owned destination ATAs into the Kamino CPI, including **withdraw_reserve_liquidity_mint**, **withdraw_reserve_collateral_mint**, **user_destination_collateral**, and **user_destination_liquidity** in [lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs](#). Those destination accounts are only validated as vault-owned ATAs in [lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs](#). After the CPI, the handler calls the proxy with an empty extras vector in [lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs](#), so the proxy never executes **ActionType::AddAsset** in [cpi_proxy.rs](#).

```
>_lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs
```

RUST

```
42: self.repay_reserve_liquidity_mint.to_account_info(),
43: self.repay_reserve_liquidity_supply.to_account_info(),
44: self.withdraw_reserve.to_account_info(),
45: self.withdraw_reserve_liquidity_mint.to_account_info(),
46: self.withdraw_reserve_collateral_mint.to_account_info(),
```

```
>_lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs
```

RUST

```
167: )]
168: pub user_source_liquidity: Box<InterfaceAccount<'info, TokenAccount>>,
169: #[account(
170:     mut,
171:     constraint = validate_vault_token_account(
```

```
>_lending_liquidate_obligation_and_redeem_reserve_collateral_v2.rs
```

RUST

```
102:         .build_remaining_accounts(ctx.remaining_accounts);
103:
104: ctx.accounts.execute_cpi(
105:     data,
106:     remaining_accounts,
```

On the pricing side, **price_vault_tokens** only prices the token accounts supplied by the caller in [price_vault_tokens.rs](#), and **validate_priced_vault_tokens** requires the priced mint set to equal **{assets U borrowable}** exactly in [vault.rs](#). As a result, if liquidation realizes a reserve mint outside that tracked set, a pricing pass that omits the mint can still succeed and understate the vault, while a pricing pass that includes it will fail until the mint is added to tracked state.

>_price_vault_tokens.rs

RUST

```
73: );  
74:  
75: for (i, chunk) in accounts_for_pricing.chunks(chunk_size).enumerate() {  
76:     let ata_info = &chunk[0];  
77:     let mint_info = &chunk[1];
```

>_vault.rs

RUST

```
34:  
35: // Instead of allocating a new set for union/assets, borrowable), check equality by co  
    mparing lengths and membership  
36: let expected_len = assets.len() + borrowable.len() - assets.intersection(&borrowable).  
    count();  
37: if expected_len != vault_assets_priced.len()  
38:     || !vault_assets_priced  
39:     .iter()
```

PRACTICAL IMPACT

If a Kamino liquidation credits a reserve liquidity or collateral mint that is not already in **{assets U borrowable}**, the vault can end up holding a real token balance that **price_vault_tokens** cannot successfully include until state is updated, understating AUM/NAV in the meantime. The risk compounds as more instructions are added that can materialize new vault-held mints, especially where those mints may not be confirmed to be priceable by the vault's supported pricing pipeline, so a fix now is highly recommended.

RECOMMENDATION

Enforce a mint-level invariant on liquidation outputs. Either reject liquidations whose realized reserve liquidity or collateral mints are outside the vault's tracked pricing set or cannot be priced by the vault's supported pricing pipeline, or add a realized output mint to **glam_state.assets** via **ActionType::AddAsset** only after confirming that subsequent vault pricing can safely include it.

FIX REVIEW NOTES

Kamino instructions now consistently gate the relevant liquidity mint.

5. Enhancement Opportunities

E01: Split oracle and execution slippage limits in `swap_handler_v2`

LOCATION

[glam/79bdd1f9/anchor/programs/glam_protocol/src/instructions/jupiter_swap_v2.rs:L283](#) 

RELEVANT SNIPPET

```
> _jupiter_swap_v2.rs RUST
281:     input_mint,
282:     output_mint,
283:     params.slippage_bps,
284: )?;
285:
```

DESCRIPTION

`swap_handler_v2` currently reuses a single `slippage_bps` value from `JupiterIxParams` for both oracle price deviation validation and post-swap execution slippage validation. In practice, managers may want a tighter oracle-deviation bound than execution slippage, or the reverse depending on market conditions and routing behavior. With the current API, they must choose a single compromise value even though the two checks protect against different risks.

POTENTIAL BENEFIT

Separate slippage limits would let managers tune quote-vs-oracle sanity checks independently from execution-vs-quote tolerance. This would make risk controls more expressive without changing the existing two-stage validation model.

RECOMMENDATION

Split the slippage controls into distinct parameters, such as `oracle_deviation_bps` and `execution_slippage_bps`. Use the oracle-specific limit in `validate_oracle_price_deviation` and the execution-specific limit in `validate_actual_slippage`. If backward compatibility matters, the new oracle-deviation limit can default to the existing `slippage_bps` until managers opt into separate configuration.

FIX REVIEW NOTES

- Oracle quote deviation now uses a separate `max_deviation_bps` policy limit, independent from execution slippage.
- This check was removed but was worth keeping as a defence in depth:
`require!(tolerance_bps <= BPS_DENOMINATOR, GlamError::PricingError);`

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

Appendix A: Hashes of program binaries

- **glam_protocol:** 52a3506dfb1bcc1c6fd76500d45d106b871077e4ee1ad31bd8528de478d1a30a
- **glam_mint:** 7bb03a50b00690dd1805b2e9cce5343d27cb16e8f6164caae1887816b9b3c44e
- **glam_config:** f9cd0a31738672c6419692a63a14edeee6d284a4bdaf116c93df2ea76e651334
- **glam_policies:** 37a98e46f74637e4cc5d4a3bdee6907bfe3373b6da4d37ef1e119ecc0248bcbc
- **ext_cctp:** 1358f3b58c4c2afdf8de884554cfe900f15e38d3cb5f8ddf5d8a7290beac361d
- **ext_kamino:** a9d944aa8bbd279be54174b878732eaf5f17278e8083f50017b11b75e73da598
- **ext_spl:** 0c201b4eb4c4bf2cd63406664ac22e23dee518556ba65bad1282a5fa93b9005f